PETWELL

PetWell API

Developer & Onboarding Documentation

A complete, first-principles guide to the NestJS backend — architecture, every module, service, guard and endpoint, and the reasoning behind each design decision.

NestJS 11 · Prisma 7 · PostgreSQL · TypeScript

Revision 2 · Backend Documentation · July 2026
Pet Health Passport & Appointment Management System

Contents

1 Introduction **5**

What problem does PetWell solve? . . . 5

Who uses it? . . . 5

How to read this document . . . 5

2 System Architecture **6**

The layers . . . 6

The life of a request . . . 6

3 Technology Stack & Key Concepts **7**

The stack . . . 7

NestJS building blocks . . . 7

DTOs, entities & data shapes . . . 7

4 Project Structure **9**

Anatomy of a feature module . . . 9

5 Application Bootstrap & Root Module **10**

main.ts — the entry point . . . 10

app.module.ts — the root module . . . 10

6 Configuration Layer **12**

env.validation.ts . . . 12

7 Database Layer **13**

The data model . . . 13

schema.prisma . . . 13

PrismaService . . . 14

DatabaseModule . . . 14

8 Infrastructure Layer **15**

BcryptService . . . 15

JwtModule . . . 15

CloudinaryService . . . 15

9 Common Layer **16**

Decorators . . . 16

RolesGuard . . . 16

Shared DTO & type augmentation . . . 16

10 Authentication & Authorisation **17**

DTOs . . . 17

login() . . . 17

AuthGuard . . . 18

11 User Module **19**

The owner address . . . 19

12 Pet Module **21**

Endpoints . . . 21

Patient lookup . . . 21

Archiving . . . 22

13 Health Record Module — the differentiator **23**

create() . . . 23

Authorisation shape . . . 23

14 Clinic Module **25**

15 Appointment Module **26**

Endpoints . . . 26

The status lifecycle . . . 26

Atomicity of status changes . . . 27

Slot capacity . . . 27

Availability . . . 29

Other guards . . . 29

16 Cross-Cutting Concerns **30**

Validation pipeline . . . 30

Error handling . . . 30

Soft-delete & audit . . . 30

Security summary . . . 30

Appendix A — Full API Reference 33

Appendix B — Glossary 35

Appendix C — Running the Project 36

1 Introduction

PetWell is a Pet Health Passport & Appointment Management System. This document explains the entire backend codebase — a REST API built with NestJS, Prisma 7 and PostgreSQL — in enough depth that a developer who has never seen the project can understand not just what the code does, but why it is written the way it is.

What problem does PetWell solve?

In the real world a pet's medical history is scattered. Each veterinary clinic keeps its own records, in its own system. When an owner visits a new clinic — or moves city — that history does not travel with them. The new vet starts blind: no vaccination dates, no allergy notes, no chronic-condition history.

PetWell fixes this with one core idea: the health record belongs to the pet and its owner, not to the clinic. Any registered clinic can add records to a pet, and the owner sees the complete history from every clinic in one place, each entry stamped with the clinic that created it. We call this a portable, clinic-agnostic health passport.

> **The differentiator in one sentence**
> Every HealthRecord stores the clinicId and createdById of whoever wrote it, so the owner's timeline can aggregate records across clinics while still showing their origin.

Who uses it?

Role	Who	Can do
OWNER	Pet owner	Manage pets, view the portable health passport, search clinics, book and cancel appointments, edit their profile.
CLINIC_STAFF	Vet / clinic employee	Look up any patient, write and update health records, manage their clinic's appointments, edit their clinic profile.

There is no admin module. Clinics are seeded into the database, and staff select their clinic at sign-up and are active immediately. This was a deliberate scoping decision: an approval workflow would have added a third role and an entire management UI without demonstrating any additional skill.

How to read this document

Chapters 2 to 4 describe the shape of the system. Chapters 5 to 9 walk the shared layers every feature depends on — bootstrap, config, database, infrastructure, common. Chapters 10 to 15 take one feature module each, in dependency order. Chapter 16 collects the concerns that cut across all of them.

Each feature chapter follows the same pattern: an overview, the request flow, the DTOs, the service logic with the interesting code quoted, then the design decisions worth defending.

2 System Architecture

The layers

Requests enter through a controller, are authorised by guards, validated by pipes, and executed by a service that owns all business logic. Only services talk to Prisma. Controllers never contain rules, and services never touch HTTP.

Layer	Responsibility	Example
Controller	Map HTTP to a service call. Thin by design.	AppointmentController
Guard	Decide whether the request may proceed.	AuthGuard, RolesGuard
Pipe	Validate and transform the payload.	Global ValidationPipe
Service	Business rules, authorisation on data, persistence.	AppointmentService
Infrastructure	Wrap a third-party concern behind an interface.	CloudinaryService, BcryptService
Database	Prisma client, one instance for the app.	PrismaService

The life of a request

Take `POST /appointments` from a signed-in owner:

1. The global **AuthGuard** runs. The route is not marked `@Public()`, so a Bearer token is required. The token is verified and its payload attached to `request.user`. 2. **RolesGuard** runs because the handler carries `@Roles(Role.OWNER)`. A clinic-staff token is rejected with 403 here. 3. The global **ValidationPipe** turns the JSON body into a `CreateAppointmentDto`, coercing `appointmentDate` to a `Date`, stripping unknown properties, and rejecting a past date. 4. **AppointmentController.create** extracts the owner id from the token and calls the service. 5. **AppointmentService.create** checks the pet, the clinic, then enforces slot capacity inside a serializable transaction and inserts the row. 6. The service returns a plain object. Nest serialises it to JSON with a 201.

The important property is that every step can fail independently and each failure maps to a specific status: 401 for a bad token, 403 for the wrong role, 400 for a bad payload, 404 for a missing pet, 409 for a full slot.

3 Technology Stack & Key Concepts

The stack

Choice	Why
NestJS 11	Opinionated structure — modules, DI, guards, pipes — which keeps a growing codebase navigable.
Prisma 7	Type-safe queries generated from one schema, and migrations that are reviewable SQL.
PostgreSQL	Relational integrity for six related entities, plus the isolation levels the booking logic depends on.
JWT (@nestjs/jwt)	Stateless auth, so the API keeps no session store.
bcrypt	Deliberately slow password hashing, salted per user.
Cloudinary	Off-loads image storage and delivery; the API only ever stores a URL.
class-validator	Declarative request validation next to the shape it validates.
zod	Validates environment variables at boot, so misconfiguration fails immediately.

NestJS building blocks

- **Module** — a unit of grouping. Each feature is a module that declares its controller and provider.
- **Controller** — a class whose methods are HTTP endpoints. It maps `POST /pets` to a method.
- **Provider / Service** — an injectable class holding logic. Injected via the constructor.
- **Guard** — returns true or throws. Runs before the handler and decides access.
- **Pipe** — transforms or validates an argument before the handler sees it.
- **Decorator** — metadata attached to a class or method, read later by a guard or interceptor.

DTOs, entities & data shapes

A DTO — Data Transfer Object — is a class that describes the shape crossing the network boundary. PetWell uses two kinds, and keeping them separate matters.

Request DTOs carry `class-validator` decorators and define what a client may send. `CreatePetDto` is the contract for creating a pet; anything not on it is stripped by the ValidationPipe.

Response DTOs are plain classes with no decorators. They describe what the API returns and, critically, they never include a password hash. `UserResponseDto` is the user minus `password`.

> **Why response DTOs at all?**
>
> They document the contract in the type system. When a service signature says `Promise<PetResponseDto[]>` you can read the return shape without tracing the Prisma query, and a field accidentally added to the database does not silently leak into the API.

4 Project Structure

```
backend/
├── prisma/
│   ├── schema.prisma          six models, four enums
│   ├── migrations/            reviewable SQL history
│   └── seed.ts                seeds the clinics
└── src/
    ├── main.ts                bootstrap
    ├── app.module.ts          root module, global AuthGuard
    ├── config/                env validation (zod)
    ├── database/              PrismaService, generated client
    ├── infrastructure/        hash, jwt, upload
    ├── common/                decorators, guards, shared DTOs
    ├── auth/                  register, login, me
    ├── user/                  profile, avatar, cover
    ├── pet/                   pet CRUD, photo, patient search
    ├── health-record/         the portable passport
    ├── clinic/                directory
    └── appointment/           booking lifecycle, capacity, audit log
```

Anatomy of a feature module

Every feature folder holds the same file kinds, which means finding your way around an unfamiliar module takes no effort:

File	Role
`*.module.ts`	Declares the controller and providers, and what it exports.
`*.controller.ts`	The HTTP endpoints. Thin — it wires request to service to response.
`*.service.ts`	All business logic and persistence for the feature.
`dto/*.dto.ts`	Request and response shapes.
`types/*.type.ts`	Local unions and helper types.
`*.constants.ts`	Tunable values and pure helpers, where a feature needs them.

5 Application Bootstrap & Root Module

main.ts — the entry point

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.useGlobalPipes(
    new ValidationPipe({
      transform: true,
      whitelist: true,
    }),
  );

  app.enableCors({
    origin: process.env.FRONTEND_URL ?? 'http://localhost:3000',
    credentials: true,
  });

  await app.listen(process.env.PORT ?? 8000);
}
```

- `transform: true` lets `@Type(() => Date)` actually convert an incoming ISO string into a `Date`, so services receive real dates rather than strings that merely look like them.
- `whitelist: true` silently drops properties not declared on the DTO. A client cannot smuggle `role: 'CLINIC_STAFF'` into a registration payload, because `RegisterDto` has no such field.
- CORS is restricted to the configured frontend origin rather than left open.

app.module.ts — the root module

```
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true, validate }),
    DatabaseModule,
    AuthModule,
    UserModule,
    PetModule,
    HealthRecordModule,
    ClinicModule,
    AppointmentModule,
  ],
  providers: [{ provide: APP_GUARD, useClass: AuthGuard }],
})
export class AppModule {}
```

The single most consequential line is `APP_GUARD`. It registers `AuthGuard` globally, which makes the API **protected by default**: every route requires a valid token unless it explicitly opts out with `@Public()`.

> **Why default-deny?**
>
> The alternative — decorating each protected route — fails silently. Forget the decorator and you have published an open endpoint with no error and no test failure. Default-deny fails in the safe direction: forget `@Public()` and a public route returns 401, which you notice immediately.

6 Configuration Layer

env.validation.ts

```
const envSchema = z.object({
  PORT: z.coerce.number().int().max(65535).min(0),
  DATABASE_URL: z.url(),
  ACCESS_TOKEN_SECRET: z.string().min(32),
  ACCESS_TOKEN_EXPIRES_IN: z.coerce.number().int().positive(),
  CLOUDINARY_CLOUD_NAME: z.string().min(1),
  CLOUDINARY_API_KEY: z.string().min(1),
  CLOUDINARY_API_SECRET: z.string().min(1),
  FRONTEND_URL: z.url().optional(),
});
```

`ConfigModule.forRoot({ validate })` runs this at boot. A missing Cloudinary key or a token secret under 32 characters stops the process with a readable error, instead of surfacing hours later as a failed upload or a forgeable token.

`z.coerce` matters because environment variables are always strings: `PORT="8000"` becomes the number 8000, and `EnvVariable` — inferred from the schema — gives `ConfigService` real types.

7 Database Layer

The data model

Six models. The relationships are what encode the product.

Model	Purpose	Key relations
User	Owners and clinic staff in one table, separated by `role`.	optional `clinic` (staff only), `pets`, `createdRecords`
Pet	The subject of the passport.	`owner`, `healthRecords`, `appointments`
HealthRecord	One clinical entry. **Carries its source clinic.**	`pet`, `clinic`, `createdBy`, optional `appointment`
Clinic	A seeded veterinary practice.	`staff`, `healthRecords`, `appointments`
Appointment	A booking request and its lifecycle.	`pet`, `clinic`, `statusLogs`
AppointmentStatusLog	One immutable status change.	`appointment`, `changedBy`

Owners and staff share the `User` table because they share every authentication concern — email uniqueness, password hashing, tokens. Only `clinicId` and `role` differ, and splitting them would have duplicated the auth module.

schema.prisma

```
model HealthRecord {
  id            String       @id @default(uuid()) @db.Uuid
  petId         String       @map("pet_id") @db.Uuid
  pet           Pet          @relation(fields: [petId], references: [id])
  clinicId      String       @map("clinic_id") @db.Uuid
  clinic        Clinic       @relation(fields: [clinicId], references: [id])
  createdById   String       @map("created_by") @db.Uuid
  createdBy     User         @relation("RecordAuthor", fields: [createdById], references: [
id])
  recordType    RecordType   @map("record_type")
  description   String
  recordDate    DateTime     @map("record_date") @db.Date
  nextDueDate   DateTime?    @map("next_due_date") @db.Date
  attachments   String[]     @default([])
  isActive      Boolean      @default(true) @map("is_active")

  @@map("health_records")
}
```

Three conventions run through the whole schema:

- `@map` **everywhere.** Prisma fields are camelCase for TypeScript; columns are snake_case for SQL. Neither language has to compromise.
- `@db.Timestamptz` **for instants,** `@db.Date` **for calendar days.** A vaccination date is a day and has no timezone; an appointment is a point in time and must have one.

- `isActive` **instead of deletion.** Nothing is hard-deleted, so history survives.

The `Appointment` model additionally carries an index that is load-bearing for the booking logic:

```
@@index([clinicId, appointmentDate])
```

Chapter 15 explains why this is a correctness concern and not merely a performance one.

PrismaService

```
@Injectable()
export class PrismaService extends PrismaClient {
  constructor(private readonly configService: ConfigService<EnvVariable, true>) {
    const adapter = new PrismaPg({
      connectionString: configService.get('DATABASE_URL', { infer: true }),
    });
    super({ adapter });
  }
}
```

Prisma 7 connects through a driver adapter — `@prisma/adapter-pg` — rather than its own query engine binary. Wrapping the client in an injectable class means services depend on `PrismaService`, not on a module-level singleton, so they stay testable.

DatabaseModule

```
@Global()
@Module({ providers: [PrismaService], exports: [PrismaService] })
export class DatabaseModule {}
```

`@Global()` means one connection pool for the process, available everywhere without each feature module importing it.

8 Infrastructure Layer

Third-party concerns live behind small services so that features depend on an interface rather than a vendor.

BcryptService

Wraps `hash` and `compare` with a fixed salt round count of 12. Because it is a provider, swapping bcrypt for argon2 touches one file.

JwtModule

Configures `@nestjs/jwt` from validated env, and `AccessTokenService` wraps signing and verification so the payload type — `AccessTokenPayload` with `sub`, `email`, `role`, `clinicId` — is enforced in one place.

Putting `clinicId` in the token is a deliberate trade. It means staff-scoped checks like "is this your clinic?" need no database round trip. The cost is that a staff member reassigned to another clinic keeps the old claim until their token expires, which is acceptable for seeded clinics that do not change.

CloudinaryService

```
upload(file: Express.Multer.File): Promise<string> {
  if (!file?.buffer) {
    throw new BadRequestException('No file was uploaded');
  }

  return new Promise((resolve, reject) => {
    const writableStream = cloudinary.uploader.upload_stream(
      (error, result) => {
        if (error || !result) {
          this.logger.error(error);
          reject(new InternalServerErrorException('Upload failed'));
          return;
        }
        resolve(result.secure_url);
      },
    );

    Readable.from(file.buffer).pipe(writableStream);
  });
}
```

Multer keeps the upload in memory, so the buffer is wrapped in a `Readable` and piped into Cloudinary's upload stream. The service returns only `secure_url`; the database stores that string and nothing else.

The guard on the first line matters more than it looks. `@UploadedFile()` is `undefined` when the multipart field is missing — a client that posts to an upload endpoint with no file attached, which happens easily by accident. Without the guard, `file.buffer` throws a `TypeError` and the client sees a 500 for what is really a bad request.

9 Common Layer

Decorators

Decorator	Purpose
`@Public()`	Marks a route as needing no token. Read by AuthGuard.
`@Roles(...)`	Declares which roles may call the handler. Read by RolesGuard.
`@CurrentUser()`	Parameter decorator returning `request.user`, or one property of it.
`@Trim()`	A `class-transformer` wrapper that trims string input.
`@TrimToNull()`	Trims, then maps an empty result to null so optional fields can be cleared.

`@CurrentUser('sub')` is worth noting: a handler that only needs the user id takes a `string` rather than the whole payload, which keeps signatures honest about what they use.

RolesGuard

```
canActivate(context: ExecutionContext): boolean {
  const requiredRoles = this.reflector.getAllAndOverride<Role[]>(ROLES_KEY, [
    context.getHandler(),
    context.getClass(),
  ]);

  if (!requiredRoles || requiredRoles.length === 0) {
    return true;
  }

  const request = context.switchToHttp().getRequest<Request>();
  const user = request.user;
  if (!user || !requiredRoles.includes(user.role)) {
    throw new ForbiddenException('Insufficient role');
  }
  return true;
}
```

It runs after the global AuthGuard, so `request.user` is already populated. It is applied per route with `@UseGuards(RolesGuard)` rather than globally, because most protected routes are open to both roles and only differ in what the service lets them see.

Shared DTO & type augmentation

`MessageResponseDto` is a one-field shape `{ message: string }` used by endpoints that only confirm an action. `src/@types/express.d.ts` augments Express's `Request` with `user?: AccessTokenPayload`, which is what makes `request.user` type-safe in guards without casting.

10 Authentication & Authorisation

> **Overview**
> Registration, login and identity. Two registration routes — one for owners, one for clinic staff — because staff additionally choose a clinic. Login returns a signed access token; everything else in the API depends on it.

DTOs

DTO	Fields
RegisterDto	firstName, lastName, phone?, email, password (min 6, alphanumeric)
RegisterClinicStaffDto	extends RegisterDto + clinicId (UUID)
LoginDto	email, password
LoginResponseDto	access_token, user (without password)

login()

```
async login(dto: LoginDto): Promise<LoginResponseDto> {
  const user = await this.userService.getUserByEmail(dto.email);
  if (!user) {
    throw new UnauthorizedException('Invalid email or password');
  }

  const isMatch = await this.bcryptService.compare(dto.password, user.password);
  if (!isMatch) {
    throw new UnauthorizedException('Invalid email or password');
  }
  if (!user.isActive) {
    throw new UnauthorizedException('Account is inactive');
  }

  const access_token = await this.accessTokenService.sign({
    sub: user.id,
    email: user.email,
    role: user.role,
    clinicId: user.clinicId,
  });

  const { password, ...rest } = user;
  return { access_token, user: rest };
}
```

- The unknown-email and wrong-password branches return the **same** message. Distinguishing them would let an attacker enumerate registered addresses.
- The password hash is destructured away before the user is returned. It never crosses the boundary.

AuthGuard

The guard reads `@Public()` first and returns early for public routes. Otherwise it requires a `Bearer <token>` header, verifies it, and attaches the payload. Its error mapping is deliberate: a malformed header is a 400, an expired token is a 401 with "Token has expired", an invalid signature is a 401 with "Invalid token". A client can tell "refresh your session" apart from "your request was wrong".

11 User Module

Profile reads and writes for whoever is signed in. There is no user-management surface — no listing users, no editing anyone else — because the product has no admin.

Endpoint	Notes
`PATCH /users/me`	Name, phone and address. Email and role are not editable.
`PATCH /users/me/avatar`	multipart `avatar`, returns the URL.
`PATCH /users/me/cover`	multipart `cover`, returns the URL.

The owner address

A Thai postal address is stored as five nullable columns rather than one free-text blob: `addressLine`, `subdistrict`, `district`, `province` and `postalCode`. Keeping the parts separate means the data stays queryable — grouping owners by province, say — which a single string never allows without re-parsing it.

Every part is optional, and sending an empty string **clears** it:

```
export function TrimToNull() {
  return Transform(({ value }: { value: unknown }) => {
    if (typeof value !== 'string') return value;
    const trimmed = value.trim();
    return trimmed === '' ? null : trimmed;
  });
}
```

> **Why clearing needs two decorators**
>
> A form that empties an input submits `''`. With a plain `@Trim()` that blank would be stored as-is, leaving the column non-null but meaningless. `@TrimToNull()` maps it to null, and `@ValidateIf((_, v) => v !== null)` then skips the format rules — without which clearing a postal code would fail the five-digit check and the field could never be removed once set.

`postalCode` is validated as exactly five digits and stored as `varchar(5)`. `province` is deliberately **not** validated against a fixed list: the frontend offers the 77 provinces as a dropdown for input quality, but pinning a geography list server-side would block a user the day an administrative name changes, for no real safety gain and no injection risk under Prisma.

The address is returned on every user response but rendered only on the owner profile. Clinic staff already have an address on their `Clinic` record, so the shared `ProfileForm` omits the section for them — and omits those keys from the request entirely, so saving a staff profile cannot blank an address that exists.

`createUser` handles two failure modes explicitly. Clinic staff without a valid `clinicId` are rejected with 400 before any write. A duplicate email is caught as Prisma error `P2002` and translated to 409 Conflict, rather than checking first — which would leave a race between the check and the insert.

> **Why upload endpoints return a raw URL**
>
> Avatar and cover upload return the URL as a bare string rather than `{ url }`, following the reference project this codebase is modelled on. Pet photo and record attachment return `{ url }`. Both are documented in Appendix A; the frontend fetch wrapper tolerates either.

12 Pet Module

> **Overview**
> Full CRUD over the owner's pets, plus the patient lookup that clinic staff use to find a pet at the counter.

Endpoints

Endpoint	Auth	Notes
`POST /pets`	OWNER	Create.
`GET /pets`	OWNER	The caller's own pets, active only.
`GET /pets/search?q=`	STAFF	Patient lookup.
`GET /pets/:petId`	Bearer	Owners see their own; staff see any.
`PATCH /pets/:petId`	OWNER	Update.
`PATCH /pets/:petId/photo`	OWNER	Cloudinary upload.
`DELETE /pets/:petId`	OWNER	Archive, not delete.

Patient lookup

```
search(query: SearchPetQueryDto): Promise<PetSearchResultDto[]> {
  const term = query.q?.trim();
  if (!term) {
    return Promise.resolve([]);
  }

  const like = { contains: term, mode: 'insensitive' as const };

  return this.prisma.pet.findMany({
    where: {
      isActive: true,
      OR: [
        { name: like },
        { microchipNo: like },
        { owner: { firstName: like } },
        { owner: { lastName: like } },
        { owner: { email: like } },
        { owner: { phone: like } },
      ],
    },
    include: {
      owner: {
        select: { id: true, firstName: true, lastName: true, phone: true, email: true },
      },
    },
    orderBy: { name: 'asc' },
    take: SEARCH_RESULT_LIMIT,
  });
}
```

A blank term returns an empty array rather than every pet, so the lookup screen opens empty instead of dumping the database. Results are capped at 50. The owner summary is included because staff need a contact detail to confirm they have the right patient.

> **Route ordering**
> `@Get('search')` is declared **before** `@Get(':petId')` in the controller. Nest matches in declaration order, so with the reverse order "search" would be captured as a `petId` and rejected by `ParseUUIDPipe` with a confusing 400.

Archiving

`DELETE /pets/:petId` sets `isActive = false`. Two consequences follow, and both are enforced:

1. An archived pet reads as absent. `findOne` throws 404 when `isActive` is false, because after a DELETE a GET should not succeed. 2. Archiving cascades to bookings. Live appointments — PENDING, CONFIRMED, RESCHEDULED — are cancelled in the same transaction as the archive, each writing an `AppointmentStatusLog` with the reason "Pet archived by owner". Without this, cancelled-by-implication appointments would sit in clinic queues forever.

13 Health Record Module — the differentiator

> **Overview**
> The portable passport. Clinic staff write records; owners read them. A record is stamped with the clinic and the author at creation, which is what lets one timeline aggregate entries from many clinics without losing provenance.

create()

```
async create(petId, user, dto): Promise<HealthRecordResponseDto> {
  if (!user.clinicId) {
    throw new ForbiddenException('Clinic staff is not linked to a clinic');
  }
  const pet = await this.prisma.pet.findUnique({
    where: { id: petId },
    select: { isActive: true },
  });
  if (!pet || !pet.isActive) {
    throw new NotFoundException('Pet not found');
  }

  return this.prisma.healthRecord.create({
    data: {
      petId,
      clinicId: user.clinicId,
      createdById: user.sub,
      recordType: dto.recordType,
      description: dto.description,
      recordDate: dto.recordDate,
      nextDueDate: dto.nextDueDate,
    },
    include: clinicInclude,
  });
}
```

`clinicId` and `createdById` come from the **token**, never from the request body. A clinic cannot attribute a record to another clinic, because the client has no way to express that.

Authorisation shape

Two private helpers carry the rules, and the asymmetry between them is the product:

- `ensurePetVisible` — used by reads. An owner may only see their own pet; staff may see any pet. This is what makes a record written at clinic A visible to clinic B.
- `ensureRecordEditable` — used by writes. Only staff **of the clinic that created the record** may edit or archive it. Clinic B can read clinic A's entry but cannot rewrite it.

> **Read widely, write narrowly**
>
> Portability requires broad reads. Trustworthiness requires narrow writes. Separating the two checks is what allows both.

Owners have no write path to health records at all — there is no owner-facing create or update route — so the passport cannot be edited by its subject.

14 Clinic Module

A read-mostly directory. Search and detail are `@Public()` so the register page can populate its clinic dropdown before anyone has a token.

Endpoint	Auth	Notes
`GET /clinics?search=&type=`	Public	Name contains, filtered by type.
`GET /clinics/:clinicId`	Public	Detail, including the Google Maps URL.
`PATCH /clinics/:clinicId`	STAFF	Own clinic only.

```
list(query: ListClinicQueryDto): Promise<ClinicResponseDto[]> {
  return this.prisma.clinic.findMany({
    where: {
      isActive: true,
      ...(query.type ? { clinicType: query.type } : {}),
      ...(query.search
        ? { name: { contains: query.search, mode: 'insensitive' } }
        : {}),
    },
    orderBy: { name: 'asc' },
  });
}
```

Conditional spreads keep optional filtering in one readable query rather than branching into several.

There is deliberately **no create and no delete**. Clinics are seeded. Deleting one would orphan the health records and appointments that reference it, and a soft-delete would still need a story for the staff accounts attached to it. Google Maps is stored as a plain URL rather than an embedded API, which keeps a key out of the project.

15 Appointment Module

> **Overview**
> The booking lifecycle. Owners request appointments; staff move them through statuses. Two invariants are defended here: an appointment's history can never disagree with its current state, and a clinic can never be over-booked for an hour.

Endpoints

Endpoint	Auth	Notes
`POST /appointments`	OWNER	Request. Enforces capacity.
`GET /appointments`	OWNER	The caller's bookings.
`GET /clinics/:clinicId/availability?date=`	Bearer	Per-hour load for a day.
`GET /clinics/:clinicId/appointments`	STAFF	Own clinic's queue.
`GET /appointments/:appointmentId`	Bearer	Detail.
`PATCH /appointments/:appointmentId/status`	STAFF	Move status, writes a log.
`DELETE /appointments/:appointmentId`	OWNER	Cancel, writes a log.
`GET /appointments/:appointmentId/logs`	Bearer	Audit trail.

The status lifecycle

Six statuses, and not every move between them is legal:

From	May become
PENDING	CONFIRMED, RESCHEDULED, REJECTED, CANCELLED
CONFIRMED	RESCHEDULED, COMPLETED, CANCELLED
RESCHEDULED	CONFIRMED, REJECTED, CANCELLED
REJECTED, COMPLETED, CANCELLED	terminal

```
private assertTransitionAllowed(from, to): void {
  if (from === to) {
    throw new BadRequestException(`Appointment is already ${to.toLowerCase()}`);
  }
  if (!ALLOWED_TRANSITIONS[from].includes(to)) {
    throw new BadRequestException(
      `Cannot change a ${from.toLowerCase()} appointment to ${to.toLowerCase()}`,
    );
  }
}
```

Owner cancellation is separately restricted to PENDING, CONFIRMED and RESCHEDULED. A completed visit cannot be retroactively cancelled.

> **Why terminal states matter**
> The audit trail is only worth having if it cannot be made to tell a false story. Allowing COMPLETED to return to PENDING, or a CANCELLED appointment to be revived, would let the log record a sequence that never happened. Rejecting same-status writes closes a second hole: padding the log with no-op entries.

Atomicity of status changes

```
const [updated] = await this.prisma.$transaction([
  this.prisma.appointment.update({
    where: { id: appointmentId },
    data: { status: dto.status },
    include: appointmentInclude,
  }),
  this.prisma.appointmentStatusLog.create({
    data: {
      appointmentId,
      changedById: user.sub,
      oldStatus: appointment.status,
      newStatus: dto.status,
      reason: dto.reason,
    },
  }),
]);
```

The transaction is the point. Either the status moves and the log records it, or neither happens. A crash between two separate writes would leave state and history disagreeing, which is exactly the corruption an audit trail is supposed to rule out.

Slot capacity

Each clinic accepts **two appointments per one-hour slot**. Slots align to the top of the hour and are half-open: the 10:00 slot covers 10:00:00 to 10:59:59.999, and 11:00 belongs to the next one. Capacity is per clinic — one clinic filling up does not affect another.

Seats are held by PENDING, CONFIRMED, RESCHEDULED and COMPLETED. CANCELLED and REJECTED release theirs, because otherwise a cancelled booking would block its slot permanently.

```
private async createWithinCapacity(dto, isRetry = false) {
  const slotStart = toSlotStart(dto.appointmentDate);
  const slotEnd = toSlotEnd(slotStart);

  try {
    return await this.prisma.$transaction(
      async (tx) => {
        const booked = await tx.appointment.count({
          where: {
            clinicId: dto.clinicId,
            status: { in: SEAT_CONSUMING_STATUSES },
            appointmentDate: { gte: slotStart, lt: slotEnd },
          },
        });
        if (booked >= MAX_APPOINTMENTS_PER_SLOT) {
          throw new ConflictException(SLOT_FULL_MESSAGE);
        }

        return tx.appointment.create({ /* ... */ });
      },
      { isolationLevel: 'Serializable' },
    );
  } catch (error) {
    if (error instanceof PrismaClientKnownRequestError &&
        error.code === WRITE_CONFLICT_CODE) {
      if (!isRetry) {
        return this.createWithinCapacity(dto, true);
      }
      throw new ConflictException(
        'That time slot is being booked by someone else. Please try again.',
      );
    }
    throw error;
  }
}
```

A full slot returns **409 Conflict** with the message "This time slot is fully booked. Please choose another time."

> **Why a plain transaction is not enough**
>
> At Prisma's default READ COMMITTED, two concurrent bookings can each count 1 and each insert, producing 3 in a slot capped at 2 — a phantom read, and the classic wrong answer to this problem. Under SERIALIZABLE, Postgres SSI detects that the count one transaction relied on was invalidated by the other and aborts it with a serialization failure, SQLSTATE 40001, surfaced by Prisma as P2034.

A write conflict does not by itself mean the slot is full — the losing transaction simply read a stale count. So the service retries once and lets the fresh count decide. Only a second conflict returns an error, and it says "try again" rather than "fully booked", because that is what actually happened.

The pet and clinic checks run **before** the transaction opens. Keeping the serializable window down to the count and the insert reduces the surface on which conflicts can occur.

> **The index is part of the correctness story**
>
> `@@index([clinicId, appointmentDate])` is not only a speed optimisation. SSI takes predicate locks over whatever the count scans. With the index it is a range scan covering just that slot; without it a sequential scan takes a table-wide lock, and bookings at unrelated clinics would spuriously abort each other.

Availability

`GET /clinics/:clinicId/availability?date=YYYY-MM-DD` returns the day's slots so a client can grey out full hours instead of letting a user submit into a wall.

```
{
  "clinicId": "...", "date": "2026-08-03", "capacityPerSlot": 2,
  "slots": [
    { "slotStart": "2026-08-03T02:00:00.000Z", "hour": 9,
      "booked": 0, "remaining": 2, "isFull": false, "isPast": true },
    { "slotStart": "2026-08-03T03:00:00.000Z", "hour": 10,
      "booked": 2, "remaining": 0, "isFull": true, "isPast": false }
  ]
}
```

The whole day is fetched in one query and bucketed in memory, which is cheaper and simpler than a grouped query per slot for a day that is at most a handful of hours wide.

Business hours are 09:00 to 18:00 local, giving nine slots with the last starting at 17:00. They live in `appointment.constants.ts` rather than on the Clinic model — a simplification that is easy to promote to a per-clinic field later.

> **Timezone**
>
> Clinics are in Thailand: ICT, UTC+07:00, with no daylight saving. Because that is a whole-hour offset, truncating an instant to the hour gives the same boundary in UTC as in local time. `slotStart` is returned as an absolute instant and should be posted straight back as `appointmentDate`; the separate `hour` field is the clinic-local hour for display, so clients never do timezone arithmetic of their own.

Other guards

`appointmentDate` must be in the future, enforced by `@MinDate(() => new Date())`. The lazy form matters: a bare `new Date()` would freeze the cutoff at module load and drift as the process ran. Archived pets cannot be booked.

16 Cross-Cutting Concerns

Validation pipeline

One global `ValidationPipe` with `transform` and `whitelist`. Every request DTO is validated the same way, and no controller repeats validation code. UUID path parameters go through `ParseUUIDPipe`, so a malformed id is a 400 before any query runs.

Error handling

Nest's built-in HTTP exceptions carry the mapping, and the choice of exception is the API contract:

Situation	Exception	Status
Bad payload, illegal status move, past date, missing file	BadRequestException	400
Missing or invalid token	UnauthorizedException	401
Authenticated but wrong role or not your resource	ForbiddenException	403
Absent, or present but not yours to see	NotFoundException	404
Duplicate email, full slot, write conflict	ConflictException	409

Cross-tenant reads return **404 rather than 403** on purpose. Telling an owner "this pet exists but is not yours" leaks the existence of another user's data; "not found" does not.

Soft-delete & audit

No row is ever removed. Pets and health records archive via `isActive`. Appointments change status and append to `AppointmentStatusLog`. Clinics have no delete path at all. The result is that every state the system passed through remains reconstructible — which is the property a health record needs most.

Security summary

Concern	Measure
Password storage	bcrypt, salt rounds 12, never returned in any response.
Transport auth	JWT Bearer tokens, expiry from validated env.
Default access	Protected by default via global AuthGuard; `@Public()` to opt out.
Authorisation	Role checks in guards, ownership and tenancy checks in services.
Mass assignment	`whitelist: true` strips undeclared properties.
User enumeration	Identical login error for unknown email and wrong password.

Concern	Measure
Existence leaks	404 instead of 403 for another tenant's resource.
Injection	Parameterised Prisma queries throughout; no raw SQL.
Configuration	zod-validated env; boot fails on a weak secret or missing key.
CORS	Restricted to the configured frontend origin.
Over-booking	Per-slot capacity enforced in a serializable transaction.

Known limitations, stated plainly: there is no refresh-token rotation, no rate limiting on login, no email verification, and no password reset. Each was scoped out rather than overlooked.

Appendix A — Full API Reference

All 31 endpoints. **Auth** column: Public means no token; Bearer means any signed-in user; OWNER or STAFF means that role only.

Method	Path	Auth	Request body / params
POST	/auth/register	Public	firstName, lastName, phone?, email, password
POST	/auth/register/clinic-staff	Public	... + clinicId
POST	/auth/login	Public	email, password
GET	/auth/me	Bearer	—
PATCH	/users/me	Bearer	firstName?, lastName?, phone?, addressLine?, subdistrict?, district?, province?, postalCode?
PATCH	/users/me/avatar	Bearer	avatar (file)
PATCH	/users/me/cover	Bearer	cover (file)
POST	/pets	OWNER	name, species, breed?, birthDate?, ...
GET	/pets	OWNER	—
GET	/pets/search	STAFF	q
GET	/pets/:petId	Bearer	—
PATCH	/pets/:petId	OWNER	(pet fields, optional)
PATCH	/pets/:petId/photo	OWNER	photo (file)
DELETE	/pets/:petId	OWNER	—
POST	/pets/:petId/health-records	STAFF	recordType, description, recordDate, nextDueDate?
GET	/pets/:petId/health-records	Bearer	—
GET	/health-records/:recordId	Bearer	—
PATCH	/health-records/:recordId	STAFF	recordType?, description?, nextDueDate?
POST	/health-records/:recordId/attachments	STAFF	file
DELETE	/health-records/:recordId	STAFF	—
GET	/clinics	Public	search?, type?
GET	/clinics/:clinicId	Public	—
PATCH	/clinics/:clinicId	STAFF	name?, address?, phone?, description?, clinicType?, googleMapsLink?
POST	/appointments	OWNER	petId, clinicId, appointmentDate, notes?
GET	/appointments	OWNER	—
GET	/clinics/:clinicId/availability	Bearer	date (YYYY-MM-DD)

Method	Path	Auth	Request body / params
GET	/clinics/:clinicId/appointments	STAFF	—
GET	/appointments/:appointmentId	Bearer	—
PATCH	/appointments/:appointmentId/status	STAFF	status, reason?
DELETE	/appointments/:appointmentId	OWNER	—
GET	/appointments/:appointmentId/logs	Bearer	—

Response shapes worth knowing

- Pet photo and record attachment upload return `{ url }`.
- Avatar and cover upload return the URL as a bare string.
- Every upload endpoint returns 400 "No file was uploaded" when the field is absent.
- Status logs include a `changedBy` summary — id, firstName, lastName, role.
- Pet search results include an `owner` summary — id, firstName, lastName, phone, email.

Appendix B — Glossary

Term	Meaning
API	The set of HTTP endpoints the backend exposes.
Bearer token	A credential sent as `Authorization: Bearer <token>`.
DTO	Data Transfer Object — a class describing a request or response shape.
Guard	A NestJS class that allows or denies a request before the handler runs.
Idempotent	An operation that has the same effect whether run once or repeated.
Isolation level	How much concurrent transactions may see of each other's work.
JWT	JSON Web Token — a signed, self-describing token.
Migration	A versioned SQL change to the database schema.
Phantom read	A row appearing mid-transaction that changes an earlier query's answer.
Pipe	A NestJS class that validates or transforms a handler argument.
Predicate lock	A lock over a *range* rather than existing rows; how SSI prevents phantoms.
Provider	Any injectable class in NestJS; services are providers.
Serializable	The strictest isolation level: results match some sequential order.
Soft delete	Marking a row inactive instead of removing it.
SSI	Serializable Snapshot Isolation — Postgres's serializable implementation.
Timestamptz	A Postgres timestamp that carries a timezone.

Appendix C — Running the Project

```
cd backend
cp .env.example .env          # DATABASE_URL, ACCESS_TOKEN_SECRET, Cloudinary keys
pnpm install
pnpm exec prisma generate     # client -> src/database/generated/prisma
pnpm exec prisma migrate dev  # create tables
pnpm db:seed                  # seed the clinics
pnpm start:dev                # http://localhost:8000
```

`ACCESS_TOKEN_SECRET` must be at least 32 characters, or boot fails by design. Generate one with:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Prisma 7 note: the datasource URL lives in `prisma.config.ts`, read from `.env`, not in `schema.prisma`. The client generates into `src/database/generated/prisma` and connects through the `@prisma/adapter-pg` driver adapter. `src/database/generated/` is gitignored, so `prisma generate` is a required step after a fresh clone.

`smoke-test.sh` in the backend folder exercises the main flows end to end against a running server with seeded accounts.